ATTS: Agentic Test-Time Scaling via Adaptive Exploration

Anonymous Author(s)
Affiliation
Address
email

Abstract

Test-time scaling has become a key lever for improving large language model reasoning, but existing approaches rely on rigid heuristic allocation or explicit search over intermediate reasoning steps. We introduce ATTS, a delegated sequential test-time scaling framework in which a lightweight orchestrator adaptively controls a pool of candidate solutions through a single action: `explore`, which launches a fresh, independent solver. The orchestrator decides when to stop exploring and directly synthesizes the final answer from the accumulated evidence. The framework defines an extensible action space over solver dispatch decisions; we instantiate it with a single parameterless `explore` tool and show that even this minimal configuration achieves strong results. We evaluate ATTS on six benchmarks spanning mathematical reasoning, scientific question answering, code generation, and multimodal reasoning. By stopping early when evidence converges, ATTS achieves competitive or superior accuracy with substantially fewer API calls than fixed-budget alternatives: 80.81% on GPQA-Diamond, 82.29% on LiveCodeBench, 56.00% on HLE-Verified, and 23.20% on BabyVision. An ablation replacing the orchestrator's direct synthesis with a separate integrator call degrades or fails to improve accuracy on three of four benchmarks, indicating that stateful evidence accumulation during exploration is critical to the method's effectiveness.

1 Introduction

Scaling inference-time compute improves large language model reasoning, but how to allocate that compute remains an open problem. No single strategy—sequential, parallel, or hybrid—is uniformly optimal across problems of varying difficulty [48], and naive scaling can saturate or even degrade accuracy through poor coverage and reward hacking [23]. The central question is not *whether* to scale, but *how* to allocate adaptively: reacting to evidence as it appears, and stopping when further exploration cannot justify its cost.

Existing approaches answer this question with either open-loop heuristics or explicit micro-level search. Predictive routing methods such as DiffAdapt use lightweight difficulty estimates to select an inference regime before generation begins, but these decisions cannot react to evidence that appears during reasoning [34]. At the other extreme, Tree of Thoughts and process-reward frameworks such as Math-Shepherd perform closed-loop adaptation through explicit tree expansion and step-level verification [58, 52]. Recent agentic test-time scaling methods add further variants, but many still rely on statistical control rules such as vote entropy or confidence-triggered halting [31, 16, 32]. An underexplored alternative is to delegate compute allocation entirely to an LLM orchestrator that makes sequential explore-or-stop decisions based on its assessment of accumulated evidence, without requiring explicit search trees or statistical stopping criteria.

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

We introduce ATTS, a **delegated sequential** test-time scaling framework that separates *control* from *solution generation*. A lightweight orchestrator manages a growing pool of candidate solutions through a single tool, `explore`, which dispatches a fresh, independent solver on the original problem. When the orchestrator judges the accumulated evidence to be sufficient, it stops exploring and synthesizes the final answer directly. ATTS treats compute allocation as closed-loop decision-making over an *extensible action space*: each `explore` call can in principle specify which solver to invoke, at what reasoning effort, and with what decomposition strategy. We evaluate the simplest instantiation—a single parameterless tool whose only decision is when to stop—and show that even this minimal agent achieves competitive accuracy at substantially lower cost. A multi-model variant that adds model selection to the action space yields further gains, supporting the principle that the orchestrator's value scales with the richness of its decision space.

We evaluate ATTS across four benchmarks spanning scientific question answering, code generation, and multimodal reasoning: HLE-Verified [60], LiveCodeBench [27], BabyVision [14], and GPQA-Diamond [43]. Because the orchestrator is benchmark-agnostic, the same policy transfers across text-only and image-conditioned tasks.

Our contributions are as follows:

1. We introduce ATTS, a delegated sequential test-time scaling framework in which a lightweight orchestrator adaptively controls a pool of independent solvers through a single action (`explore`), deciding when to stop based on accumulated evidence.
2. We evaluate ATTS on six diverse benchmarks and show that it achieves competitive or superior accuracy to fixed-budget baselines while using substantially fewer API calls—80.81% on GPQA-Diamond, 82.29% on LiveCodeBench, and 56.00% on HLE-Verified.
3. Through ablations we show that the orchestrator's stateful evidence accumulation is critical: replacing direct synthesis with a separate integrator degrades accuracy on three of four benchmarks, and the framework generalizes across model families and action-space configurations.

2 Related Work

Test-Time Scaling. Inference-time compute for LLMs is commonly organized along three axes [48]. *Sequential scaling* increases reasoning depth within a single trajectory, including self-correction [37], budget-controlled decoding [39], and long-thinking models that internalize extended chains of thought via reinforcement learning [21]. A critical negative result is that LLMs cannot self-correct reasoning through prompting alone [24], motivating architectures that separate evaluation from solution generation. *Parallel scaling* increases reasoning width by sampling multiple independent candidates and aggregating them. Self-consistency marginalizes over diverse reasoning paths [54]; coverage scales log-linearly with the sample count [11]; and Universal Self-Consistency uses the LLM itself to select among candidates for free-form tasks [15]. Learned reward models and verifiers improve selection beyond majority voting [48, 33, 51, 53], and Multi-Sequence Verifiers jointly process the entire candidate pool rather than scoring each independently [6]. Theoretical analysis shows that naive parallel scaling is not uniformly beneficial: overly aggressive sampling can degrade accuracy through reward hacking [23, 13]. *Hybrid search* combines depth and width by expanding and pruning partial trajectories via tree search [58], Monte Carlo Tree Search with preference learning [57], step-level process reward models [52], or interleaved aggregation and refinement [50]. ATTS is neither a single-trajectory method nor a tree-search method: it delegates *independent complete solves* to a separate explorer and manages the resulting candidate pool sequentially, occupying a distinct point in this design space.

Adaptive Compute Allocation. A growing body of work addresses *when* to stop sampling and *how much* compute to allocate per problem. Statistical stopping criteria include Beta/Dirichlet confidence thresholds [2], the Sequential Probability Ratio Test [30], anytime-valid concentration bounds via e-processes [18], Bayesian optimal stopping with conjugate priors [1], and optimal answer tracking showing that monitoring the top-3 answers suffices for asymptotic optimality [63]. Kalayci et al. [29] frame each generation as opening a box in Weitzman's Pandora's Box problem, deriving reservation-price stopping rules. On the learned side, Damani et al. [19] train a difficulty predictor to adapt the number of best-of-k samples per query, and BEST-Route jointly selects which model and

how many responses to sample [44]. These methods rely on either statistical criteria or learned scalar predictors. At the theoretical level, De Sabbata et al. [20] operationalize rational metareasoning for LLMs at the *token level*, training models with a VOC-derived reward that penalizes unnecessary reasoning steps within a single generation; they identify candidate-level agentic extension as an open direction. ATTS uses an LLM orchestrator that reads full reasoning traces—accessing richer information than answer frequencies or scalar scores—within a sequential closed loop where each observation informs the next action.

Agentic Orchestration. Recent work studies both agentic test-time scaling and learned meta-level controllers. On the scaling side, CATTS allocates compute via vote-derived uncertainty [31], TUMIX coordinates parallel heterogeneous agents [16], and General AgentBench benchmarks sequential and parallel scaling for open-ended agents [32]. On the meta-control side, Manvi et al. [38] show that LLMs can predict mid-generation whether restarting will help; CoRefine attaches a lightweight controller that decides to halt, re-examine, or try a different approach [5]; Meta-Reasoner employs a contextual bandit over high-level strategies such as backtrack, restart, or switch approach [3]; AB-MCTS dynamically decides between expanding width and depth [26, 9]; and ORCH orchestrates multiple LLM analyses followed by a single merge [62]. A parallel trend trains LLM orchestrators via reinforcement learning: Router-R1 uses GRPO for multi-round routing to external LLMs [61], xRouter learns cost-aware orchestration across 20+ models [42], and Search-R1 trains multi-turn search-interleaved reasoning [28]. On multi-agent interaction, symmetric debate has been shown to induce a martingale over belief trajectories, providing no expected accuracy improvement beyond majority voting [17, 56]. ATTS differs from these approaches in one key respect: it dispatches *fresh independent solvers* (not routing to different models or managing a single evolving trajectory), and only the orchestrator accumulates evidence—an asymmetric, delegated architecture with a single action type (`explore`) whose value is assessed after each invocation.

Cost-Aware Agent Training. A nascent line of work incorporates execution costs into agent objectives. CATP-LLM optimizes a performance-cost tradeoff for tool planning via cost-aware offline RL [55]. Calibrate-Then-Act formalizes the cost-uncertainty tradeoff for agent exploration, comparing the expected information value of an additional tool call against its cost [4]. On reward engineering for tool use, ToolRL systematically studies reward type, scale, and granularity for GRPO-based training [41]; StepTool assigns per-interaction rewards based on invocation success and task contribution [59]; and the Tool-call Reward Model enables step-level credit assignment in PPO/GRPO pipelines [7]. These works address planning, exploration, and reward design for general tool use. ATTS targets a more specific problem: *candidate-level test-time scaling*, where the "tool" is an independent solver call and the meta-decision is a sequential explore-or-stop choice. To our knowledge, no prior work combines an LLM orchestrator that reads full candidate reasoning traces, sequential closed-loop stopping, and independently generated candidates within a unified framework for test-time compute allocation.

3 Methodology

Test-time scaling is a resource allocation problem: given a budget of model calls, how should an agent decide what to compute next and when to stop? Drawing on the principle of *rational metareasoning* [45, 22]—that an agent should reason about whether additional computation justifies its cost—we design ATTS as a delegated sequential framework in which a lightweight orchestrator manages a growing pool of candidate solutions, decides when to stop exploring, and synthesizes the final answer from the accumulated evidence.

3.1 The ATTS Loop

The orchestrator repeatedly decides whether to call `explore` or to stop and synthesize. It *never* solves the problem itself.

Explore. Each `explore` call dispatches a fresh solver on the original problem x. The solver works *from scratch*—it does not see or continue previous candidates—and returns a structured candidate $c_t = (\text{answer}, \text{reasoning}, \text{approach}, \text{confidence})$ that is appended to $\mathcal{C}_t$. By construction, each explorer receives only the problem statement, guaranteeing that explore results are *conditionally*

Algorithm 1 ATTS: Adaptive Test-Time Scaling

Require: Problem x, exploration budget T, orchestrator LLM $\mathcal{O}$, solver LLM $\mathcal{S}$
Ensure: Final answer y

```
C ← ∅ {candidate pool}
for t = 1 to T do
    action ← O(x, C) {decide: explore or stop}
    if action = stop then
        break
    end if
    c_t ← S(x) {fresh, independent solve}
    C ← C ∪ {c_t}
end for
y ← O(x, C) {synthesize final answer}
return y
```

independent given the problem. This independence means the orchestrator can assess the marginal contribution of each additional explore in isolation, without modeling interactions between future computations.

Stopping. The orchestrator can stop at any point before the exploration budget T is exhausted. The stopping decision is made by the orchestrator model itself based on its assessment of the candidate pool. The prompt encodes principles for this judgment: genuine convergence means independent solvers arriving at the same answer through different methods, repeated failures indicate the problem exceeds solver capability, and each additional explore call has a cost that must be justified by expected information gain. This judgment reflects the same cost-benefit tradeoff formalized by the *value-of-computation* criterion [22]: additional exploration is warranted only when the expected information gain justifies the cost.

Synthesis. When the orchestrator stops, it synthesizes the final answer directly from $\mathcal{C}_t$. Because it has observed the full multi-turn history—each candidate's arrival, its reasoning, and its own deliberation about whether to continue—it has richer context than a separate aggregator that only sees the final pool.

Algorithm 1 summarizes the procedure. The control logic is benchmark-agnostic: only the explorer prompt and answer schema change across tasks. The orchestrator prompt encodes its decision logic as declarative principles (e.g. "a single candidate does not constitute sufficient evidence") rather than prescriptive rules, paired with worked examples that demonstrate how these principles apply in concrete scenarios. Appendix A gives the full implementation details including representative prompts.

3.2 Extensible Action Space

The value of the framework scales with the richness of the orchestrator's action space. In the simplest instantiation, the only action is "explore once more with the same model," and the orchestrator can only decide *when* to stop. With heterogeneous computation types, it also decides *what* to compute—selecting among different effort levels, solver models, or time budgets based on the current evidence. Each extension adds degrees of freedom for adapting to problem difficulty without changing the underlying loop.

This yields a concrete design principle: ATTS's architecture is not a fixed pipeline but a framework for adaptively selecting what to compute at test time, in the spirit of Hay et al. [22]. The experiments in this paper evaluate the simplest instantiation (single model, uniform effort) alongside a multi-model extension with configurable exploration effort (§4).

3.3 Multi-Model Extension

We extend ATTS to multi-model deployment by adding model selection to the orchestrator's decision space. The orchestrator (Claude Sonnet 4.6) dispatches explores to a pool of three models—

Haiku (cheapest), Sonnet (mid-range), and Opus (strongest)—each with a per-question budget limit. Model profiles inform dispatch decisions: the orchestrator starts with cheaper models, escalates on disagreement or repeated failure, and treats cross-model agreement as stronger evidence than same-model convergence. This enables cost-aware exploration strategies where easy problems are solved entirely with cheap models while hard problems automatically escalate to more capable solvers. An *exploration effort* parameter controls the exploration policy: Low effort instructs the orchestrator to stop as soon as any two candidates converge, Medium effort requires at least three explores across at least two different models before the orchestrator's stopping judgment takes effect, and High effort requires at least five explores using all three models.

4 Experiments

We evaluate on four benchmarks: HLE-Verified [60], LiveCodeBench [27], BabyVision [14], and GPQA-Diamond [43], which collectively span scientific question answering, code generation, and multimodal reasoning. Appendix E collects the full official task definitions, grading rules, source repositories, and leaderboard alignment notes for all benchmarks.

4.1 Experimental Setup

Across all experiments, the underlying backbone model is Claude Sonnet 4.6 (`claude-sonnet-4-6`), used for the orchestrator, explorer, and all baselines unless otherwise noted. Claude Haiku, Sonnet, and Opus are three capability tiers in the Claude model family, ordered from smallest/cheapest to largest/most capable. All generation uses the model's default sampling parameters (no explicit temperature or top-p override) with extended thinking enabled at a budget of 32,000 tokens and the API's default reasoning effort (no explicit `effort` override). Extended thinking allows the model to allocate a portion of its output budget to internal chain-of-thought reasoning before producing the final structured response; all baselines use the same setting for fair comparison. The exploration budget is $T{=}8$ `explore` calls per question for single-model runs and $T{=}16$ for multi-model runs (per-model limits: Haiku 8, Sonnet 8, Opus 4).

We compare against six baselines, all sharing the same backbone model and answer interface so that only the inference-time strategy differs (full details in Appendix A): Pass@1 (single rollout), Majority Voting ($N{=}8$ independent candidates, most-frequent answer; reported only on benchmarks where direct answer aggregation is well-defined), Self-Refine [37] (iterative feedback-revision on a single draft), Budget Forcing [39] (N sequential rounds with forced continuation via "Wait"), reward-model reranking using Skywork-Reward-V2 [33, 47] for text benchmarks or Visual-PRM [53, 40] for multimodal benchmarks, and LLM Selection ($N{=}8$), which feeds all pre-cached candidates to the backbone model in a single call and asks it to analyze, compare, and synthesize a final answer. LLM Selection isolates the contribution of LLM-based aggregation from adaptive sequential exploration.

For ATTS, the controller issues up to the exploration budget of `explore` calls before stopping and synthesizing the final answer directly, as described in Section 3. The comparison across all rows isolates differences in test-time inference strategy rather than differences in training data, parameter count, or post-training recipe.

Cost accounting. All costs reported as \$/q reflect API token charges only, computed from per-model pricing and recorded token usage. For ATTS, this includes orchestrator tokens, explorer tokens, and (when applicable) integrator tokens. For reward-model reranking, the reported cost covers the $N{=}8$ candidate generation calls; the reward model itself runs locally and incurs no API cost but does require GPU compute not reflected in the dollar figure. Grading costs (LLM judge calls for HLE and BabyVision) are excluded from all rows. In all tables, **bold** marks the best accuracy per group and the lowest \$/q among test-time scaling methods.

Run-to-run stability. To assess variance, we run ATTS five times on GPQA-Diamond with different random seeds (varying the order of questions and orchestrator sampling). The results are highly stable: $80.20\% \pm 0.55\%$ (range: 79.8%–80.8%, $\sigma = 0.55$), with cost $\$0.32 \pm \$0.01$/q. The standard deviation of 0.55 percentage points corresponds to a difference of approximately two questions across runs, indicating that the orchestrator's stopping and synthesis decisions are robust to

Table 1: Main results on four benchmarks. Top left: HLE-Verified (Accuracy). Top right: LiveCodeBench (Pass@1, overall and by difficulty). Bottom left: GPQA-Diamond (Accuracy). Bottom right: BabyVision (Avg@3, overall and by task family). Each subtable shows zero-shot model references (upper) and test-time scaling methods (lower). **Bold** marks the best per group; lowest $/q among test-time methods is also bolded.

(a) HLE-Verified

	Acc. (%)	$/q
Zero-Shot Models		
Gemini 3 Pro	48.93	–
GPT-5.2-High	**52.48**	–
Claude Opus 4.5	48.16	–
Grok 4.1 Fast Reasoning	43.07	–
Claude Opus 4.6	50.16	–
DeepSeek-V3.2	47.45	–
Qwen3-Max-Thinking	48.48	–
Test-Time Scaling Methods		
Pass@1	48.00	0.50
Self-Refine	53.00	**0.41**
Budget Forcing	51.00	0.60
Skywork-Reward-V2	52.00	3.61
LLM Selection (N=8)	**58.00**	3.71
ATTS (Ours)	56.00	1.59
ATTS Multi-model Med	57.00	1.54

(b) LiveCodeBench

	Pass@1	Easy	Med.	Hard	$/q
Zero-Shot Models					
O4-Mini (High)	**80.2**	**99.1**	**89.4**	**63.5**	–
O3 (High)	75.8	**99.1**	84.4	57.1	–
O4-Mini (Med.)	74.2	98.2	86.5	52.7	–
Gemini-2.5-Pro	73.6	**99.1**	87.2	50.2	–
DeepSeek-R1	73.1	98.7	85.2	50.7	–
Test-Time Scaling Methods					
Pass@1	77.14	95.35	88.46	60.00	**0.24**
Self-Refine	80.57	**100.00**	90.38	63.75	0.32
Budget Forcing	80.00	**100.00**	88.46	63.75	0.50
Skywork-Reward-V2	78.86	**100.00**	90.38	60.00	1.41
LLM Selection (N=8)	81.14	**100.00**	86.54	67.50	1.51
ATTS (Ours)	82.29	**100.00**	90.38	67.50	0.53
ATTS Multi-model Med	**85.63**	**100.00**	**92.31**	**73.42**	0.69

(c) GPQA-Diamond

	Acc. (%)	$/q
Zero-Shot Models		
Gemini 3.1 Pro Preview	**94.1**	–
GPT-5.4 (xhigh)	92.0	–
GPT-5.3 Codex (xhigh)	91.5	–
Gemini 3 Pro Preview (high)	90.8	–
GPT-5.2 (xhigh)	90.3	–
Test-Time Scaling Methods		
Pass@1	74.24	**0.13**
Majority Voting	73.74	0.90
Self-Refine	73.10	0.23
Budget Forcing	77.16	0.42
Skywork-Reward-V2	76.14	0.90
LLM Selection (N=8)	77.16	0.97
ATTS (Ours)	80.81	0.34
ATTS Multi-model Med	**81.82**	0.39

(d) BabyVision

	Overall	Fine-gr.	Tracking	Spatial	Pattern	$/q
Human	94.1	92.3	94.6	94.7	97.8	–
Zero-Shot Models						
Gemini3-Pro-Preview	**49.7**	**46.2**	**43.4**	**53.7**	53.9	–
GPT-5.2	34.4	27.3	34.9	35.2	**54.9**	–
Doubao-1.8	30.2	39.2	15.7	24.7	37.7	–
Qwen3-VL-Plus	19.2	21.8	11.5	18.1	25.5	–
Grok-4	16.2	11.0	24.1	13.2	24.8	–
Test-Time Scaling Methods						
Pass@1	19.59	20.86	19.28	19.78	15.69	**0.07**
Majority Voting	**23.20**	**23.93**	21.69	23.08	**23.53**	0.50
Self-Refine	21.39	20.25	22.89	20.88	**23.53**	0.42
Budget Forcing	21.91	20.86	**28.92**	21.98	13.73	0.52
VisualPRM	20.88	20.86	21.69	21.98	17.65	0.50
LLM Selection (N=8)	**23.20**	23.31	25.30	23.08	19.61	0.56
ATTS (Ours)	**23.20**	22.09	24.10	**26.37**	19.61	0.27
ATTS Multi-model Med	22.42	22.09	27.71	21.98	15.69	0.43

sampling stochasticity. All other results in the paper are single-run evaluations; the GPQA stability analysis suggests that the reported numbers are representative.

4.2 ATTS Achieves Competitive Accuracy at Lower Cost

Table 1 presents the main results across all four benchmarks. For each benchmark, the upper group reproduces published zero-shot references for context, and the lower group reports test-time scaling methods evaluated under our unified setup. Zero-shot references are cited from the respective benchmark papers and official leaderboards; they use different model configurations and are therefore not directly comparable to our test-time scaling rows. HLE-Verified zero-shot rows report Revised-subset Acc [60] while our rows evaluate a 100-sample Gold subset. LiveCodeBench references follow the public generation leaderboard [27]. BabyVision references are from Chen et al. [14]. GPQA-Diamond references are from Rein et al. [43] and official records. Full benchmark details including dataset releases, grading protocols, and subset definitions are in Appendix E.

HLE-Verified. On HLE-Verified, ATTS achieves 56.00% accuracy at $1.59/q, outperforming Self-Refine (53.00%) and Budget Forcing (51.00%). LLM Selection reaches 58.00% but at $3.71/q—a 2.3× cost increase for a 2-point gain—illustrating that single-pass aggregation achieves higher accuracy when cost is unconstrained, but adaptive exploration provides a more efficient operating point. The multi-model variant improves to 57.00% at $1.54/q, narrowing the accuracy gap while maintaining cost efficiency.

LiveCodeBench. For LiveCodeBench, the single-model ATTS configuration reaches 82.29% Pass@1 at $0.53/q, improving over Self-Refine (80.57%) and Budget Forcing (80.00%). The gains concentrate on Hard problems: 67.50% vs. 63.75% for both Self-Refine and Budget Forcing, while Easy (100%) and Medium (90.38%) accuracies are at or near ceiling. The multi-model variant further improves to 85.63% overall and 73.42% on Hard.

GPQA-Diamond. On GPQA-Diamond, ATTS achieves the highest accuracy among all test-time scaling methods (80.81%) at $0.34/q, outperforming Budget Forcing (77.16%) and Skywork-Reward-V2 (76.14%). Self-Refine (73.10%) falls *below* Pass@1 (74.24%), consistent with the known limitation that its feedback model frequently accepts the initial draft and occasionally introduces errors during revision.

BabyVision. Among test-time scaling methods, ATTS ties with Majority Voting and LLM Selection at 23.20% overall accuracy on BabyVision, but at $0.27/q—roughly half the cost of either alternative. The largest per-subset margin appears on Spatial Perception, where ATTS reaches 26.37%.

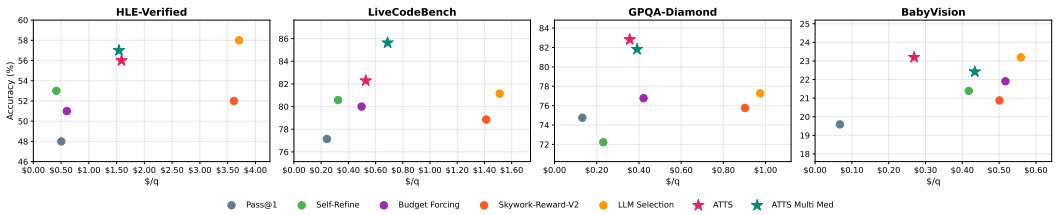

Figure 1: Cost-accuracy tradeoffs across the four main benchmarks (left to right: HLE-Verified, LiveCodeBench, GPQA-Diamond, BabyVision). Star markers denote ATTS variants; circles denote baselines. ATTS achieves competitive or superior accuracy at lower cost across all benchmarks.

Overall trends. Across all four benchmarks, ATTS achieves the best or tied-best accuracy among test-time scaling methods while costing 1.7–2.3× *less* than alternatives of comparable accuracy. The gains are largest where one-shot generation leaves the most headroom: on GPQA-Diamond, ATTS improves over Pass@1 by 6.6 percentage points (74.24% → 80.81%) at $0.34/q; on LiveCodeBench Hard problems, accuracy rises from 60.00% to 67.50%. On easy subtasks where the backbone is already near ceiling (100% Easy on LiveCodeBench), additional exploration yields no benefit—and ATTS does not waste budget there. Where a higher-cost method edges ahead in accuracy—LLM Selection reaches 58.00% on HLE versus ATTS's 56.00%—it does so at 2.3× the cost. This pattern reflects the adaptive allocation at the core of the framework: explore when the current candidate pool is incomplete or inconsistent, and synthesize once the evidence suffices. The results support our main claim: even a purposefully constrained single-tool interface can turn test-time scaling into a semantic compute-allocation problem rather than a fixed sampling schedule.

4.3 Ablation Studies

Medium effort is the best cost-efficiency operating point. The multi-model variant exposes an *exploration effort* parameter that controls stopping aggressiveness. Figure 2 shows the cost-accuracy tradeoff at three effort levels (Low, Medium, High) across all four benchmarks, holding the model pool and budget fixed; the raw numbers are in Appendix B, Table 5. The Low-to-Medium transition yields the largest accuracy gain on every benchmark (up to +13 points on HLE, +8.6 on LiveCodeBench). Medium-to-High provides diminishing returns: on LiveCodeBench and BabyVision accuracy is flat while cost nearly doubles; only on HLE-Verified does High effort produce a meaningful further improvement (+3 points). Medium effort therefore represents the best cost-efficiency operating point across the benchmark suite.

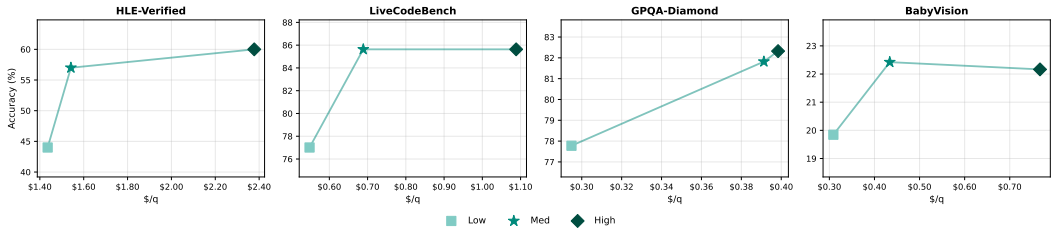

Figure 2: Effect of exploration effort on multi-model ATTS. Each panel shows the cost-accuracy tradeoff for Low, Medium, and High effort on one benchmark. The steep Low-to-Medium gain and flat Medium-to-High plateau are consistent across all benchmarks.

Stateful orchestration outperforms separate integration. In the default ATTS architecture, the orchestrator produces the final answer directly from its accumulated evidence after deciding to stop exploring. We also evaluate a variant that adds a dedicated `integrate` step: a separate model call that receives all candidates and synthesizes the final answer independently.

Table 2: Effect of adding a separate integrator. The default ATTS architecture has the orchestrator synthesize directly; the "+ Integrator" variant delegates synthesis to a separate model call.

	Default (no integrator)		+ Integrator	
	Acc. (%)	$/q	Acc. (%)	$/q
HLE-Verified	56.00	1.59	57.00	2.35
LiveCodeBench	82.29	0.53	80.57	0.72
BabyVision	23.20	0.27	23.20	0.38
GPQA-Diamond	80.81	0.34	76.14	0.44

Across all four benchmarks, adding a separate integrator does *not* consistently improve accuracy and *always* increases cost (Table 2). On GPQA-Diamond, accuracy drops from 80.81% to 76.14%; on LiveCodeBench, from 82.29% to 80.57%. On HLE-Verified, the integrator variant gains 1 point (57.00% vs. 56.00%) but at substantially higher cost ($2.35/q vs. $1.59/q). The degradation likely stems from the integrator having *less* context for synthesis: it only sees the final candidate pool in a single-turn call, whereas the orchestrator has observed the full multi-turn conversation including each candidate's arrival and the reasoning behind its own exploration decisions.

Orchestrator effectiveness is task-dependent, not capability-monotonic. We ablate the orchestrator model while keeping the explorer fixed at Sonnet 4.6 (Table 6, Figure 3). The results show no monotonic relationship between model strength and orchestrator effectiveness: Opus achieves the highest accuracy on HLE (59%) and LiveCodeBench (83.4%), but drops to 77.16% on GPQA—*below* Sonnet's 80.81%. Haiku performs comparably to Sonnet on HLE (57% vs. 56%).

These differences are not explained by stopping behavior. All three orchestrators use similar numbers of explores on each benchmark (e.g. 2.15–2.43 on GPQA) and read from the same candidate cache. The gap is therefore attributable to synthesis quality: given similar evidence, the orchestrators produce different final answers. The pattern is *benchmark-dependent* rather than *capability-monotonic*—the strongest model is the best orchestrator on some benchmarks but the *worst* on others. This indicates that orchestrator effectiveness depends on a task-specific interaction between the model's reasoning style and domain characteristics, not simply on model capability. Identifying the precise mechanism underlying this interaction remains an open question.

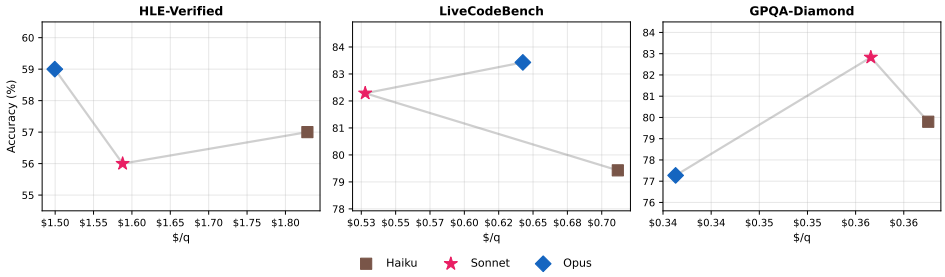

Figure 3: Effect of orchestrator model on ATTS accuracy and cost. Haiku is cheapest per call but over-explores, increasing total cost. Opus improves accuracy on HLE and LiveCodeBench but not on GPQA.

The framework generalizes across model families. To evaluate whether ATTS generalizes beyond a single model family, we run the full framework using GPT-5.2 (OpenAI) as the backbone for all three roles—orchestrator, explorer, and synthesizer—keeping the same exploration budget (T=8) and prompts as the Claude Sonnet configuration. We evaluate on two benchmarks (HLE-Verified and GPQA-Diamond) and test two reasoning effort levels to assess the effect of orchestrator reasoning depth.

Table 3: Backbone model ablation. All configurations use the default ATTS architecture (no separate integrator) with identical prompts and exploration budget. Reasoning effort controls the orchestrator's internal chain-of-thought budget. ATTS gain = ATTS accuracy − Pass@1 accuracy.

Backbone	Effort	Benchmark	Pass@1	ATTS	Gain	$/q
Claude Sonnet 4.6	default	HLE	48.00	56.00	+8.0	1.59
Claude Sonnet 4.6	default	GPQA	74.24	80.81	+6.6	0.34
GPT-5.2	high	HLE	55.00	57.00	+2.0	0.26
GPT-5.2	high	GPQA	56.06	**71.07**	**+15.0**	**0.16**

ATTS produces positive gains on both benchmarks for both model families (Table 3). On GPQA-Diamond, GPT-5.2 shows the largest absolute gain: +15.0 points over Pass@1 (56.06% → 71.07%), compared to Claude Sonnet's +6.6 points. The lower GPT-5.2 Pass@1 (56.06% vs. the 90.3% reported for GPT-5.2 in the GPQA leaderboard) reflects the use of `effort=low` for exploration, which limits reasoning tokens; the orchestrator operates at `effort=high` during synthesis. On HLE-Verified, the gain is smaller (+2.0 points) but still positive, and the cost is $6.1\times$ lower than Claude. These results demonstrate that ATTS's adaptive exploration framework is not specific to any model family: the same prompts and control logic transfer to a different provider with consistent gains at substantially lower cost.

Orchestrator reasoning contributes to synthesis quality. We ablate the orchestrator's internal reasoning by disabling extended thinking (chain-of-thought) while keeping all other parameters identical: same model (Sonnet 4.6), same cached explores, same exploration budget (T=8), and same prompts. Without extended thinking, the orchestrator can still read candidate results, call tools, and submit answers—but cannot reason internally between turns.

Table 4: Effect of disabling orchestrator extended thinking on GPQA-Diamond. All other parameters are held constant. The orchestrator uses the same cached Sonnet explores in both conditions.

Condition	Acc. (%)	$/q	Explores/q
Default (thinking enabled)	82.83	0.35	2.15
Thinking disabled	79.29	0.36	2.08

Disabling thinking reduces accuracy by 3.5 points (82.83% → 79.29%) with no cost savings (Table 4). The exploration behavior is nearly identical (2.08 vs. 2.15 explores per question), indicating

that the accuracy gap is attributable to synthesis quality, not stopping behavior. This result confirms that the orchestrator's internal reasoning—even when constrained to a pure orchestration role with no direct problem-solving—contributes meaningfully to evidence evaluation and answer selection.

5 Limitations

ATTS has four principal limitations. First, it can only synthesize evidence present in the candidate pool: if all candidates share the same misconception, the orchestrator cannot recover. Second, the framework provides no formal guarantees for stopping optimality, calibration, or budget efficiency; the orchestrator remains a learned heuristic. In practice, the orchestrator converges early: on GPQA, 82% of questions use exactly two `explore` calls before stopping, even though the oracle best-of-N curve continues to climb through $N{=}8$. This conservative stopping leaves accuracy on the table, and learning a better stopping criterion remains an open problem. Third, ATTS increases latency and cost relative to direct generation on easy problems where exploration is unnecessary. Finally, the evaluation covers reasoning benchmarks with fixed answer interfaces, not open-ended tool-use agents; the results are evidence about reasoning-oriented test-time scaling, not a complete account of agentic decision-making. We additionally note that on a small fraction of questions (6 out of 860 across the four benchmarks; see Figure 4) the orchestrator produces an answer without issuing any `explore` call, despite the prompt directing it to invoke `explore` before synthesizing; the prompt is designed to suppress this behavior and the residual exceptions do not materially affect the aggregate results.

6 Conclusion

The central empirical finding of this work is that an LLM orchestrator performing closed-loop decision-making over a minimal action space—a single `explore` tool—achieves competitive or superior accuracy to fixed-budget test-time scaling strategies while using substantially fewer API calls. A multi-model variant further improves to 57.00% and 85.63% on HLE-Verified and Live-CodeBench respectively. The ablation results sharpen this finding: removing the orchestrator's multi-turn context by delegating synthesis to a separate call degrades or fails to improve accuracy on three of four benchmarks, demonstrating that *stateful evidence management*—not just candidate diversity—drives the gains. For the broader field, these results indicate that test-time compute allocation is better approached as closed-loop decision-making by an LLM agent—with an extensible action space over solver dispatch—than as explicit tree search or fixed sampling. A key limitation is the orchestrator's conservative stopping behavior: on GPQA-Diamond, 82% of questions use exactly two `explore` calls, leaving substantial headroom relative to oracle best-of-N. Learning stopping policies that better exploit the full exploration budget remains an open problem. Code and cached experimental artifacts are available at `https://github.com/t2ance/ATTS`.

References

[1] BEACON: Bayesian optimal stopping for efficient LLM sampling, 2025. URL `https://arxiv.org/abs/2510.15945`.

[2] Pranjal Aggarwal, Aman Madaan, Yiming Yang, and Mausam. Let's sample step by step: Adaptive-consistency for efficient reasoning and coding with LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12375–12396. Association for Computational Linguistics, 2023. URL `https://aclanthology.org/2023.emnlp-main.761/`.

[3] Anonymous. Meta-reasoner: Dynamic guidance for optimized inference-time reasoning in large language models, 2025. URL `https://arxiv.org/abs/2502.19918`.

[4] Anonymous. Calibrate-then-act: Cost-aware exploration in LLM agents, 2026. URL `https://arxiv.org/abs/2602.16699`.

[5] Anonymous. CoRefine: Confidence-guided self-refinement for adaptive test-time compute, 2026. URL `https://arxiv.org/abs/2602.08948`.

[6] Anonymous. Parallel test-time scaling with multi-sequence verifiers, 2026. URL https://arxiv.org/abs/2603.03417.

[7] Anonymous. Empowering LLM tool invocation with tool-call reward model, 2026. URL https://openreview.net/forum?id=LnBEASInVr. Under review.

[8] Anthropic. Claude code SDK documentation. https://docs.anthropic.com/en/docs/claude-code/sdk, 2025.

[9] Ahsan Bilal, Ahmed Mohsin, Muhammad Umer, Ali Subhan, Hassan Rizwan, Ayesha Mohsin, and Dean Hougen. What if we allocate test-time compute adaptively?, 2026. URL https://arxiv.org/abs/2602.01070.

[10] David Blackwell. Equivalent comparisons of experiments. *The Annals of Mathematical Statistics*, 24(2):265–272, 1953.

[11] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL https://arxiv.org/abs/2407.21787.

[12] Center for AI Safety, Scale AI, and HLE Contributors Consortium. A benchmark of expert-level academic questions to assess AI capabilities. *Nature*, 649:1139–1146, 2026. doi: 10.1038/s41586-025-09962-4.

[13] Feng Chen, Allan Raventos, Nan Cheng, Surya Ganguli, and Shaul Druckmann. Rethinking fine-tuning when scaling test-time compute: Limiting confidence improves mathematical reasoning. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2502.07154.

[14] Liang Chen, Weichu Xie, Yiyan Liang, et al. Babyvision: Visual reasoning beyond language, 2026. URL https://arxiv.org/abs/2601.06521.

[15] Xinyun Chen, Renat Aksitov, Uri Alon, Jie Ren, Kefan Xiao, Pengcheng Yin, Sushant Prakash, Charles Sutton, Xuezhi Wang, and Denny Zhou. Universal self-consistency for large language model generation. In *ICML 2024 Workshop on In-Context Learning*, 2024. URL https://arxiv.org/abs/2311.17311.

[16] Yongchao Chen, Jiefeng Chen, Rui Meng, Ji Yin, Na Li, Chuchu Fan, Chi Wang, Tomas Pfister, and Jinsung Yoon. Tumix: Multi-agent test-time scaling with tool-use mixture, 2025. URL https://arxiv.org/abs/2510.01279.

[17] Hyeong Kyu Choi and Banghua Zhu. Debate or vote: Which yields better decisions in multi-agent large language models? In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2508.17536.

[18] Abel Cordero-Encinar and Andrew Duncan. Certified self-consistency: Statistical guarantees and test-time training, 2025. URL https://arxiv.org/abs/2510.17472.

[19] Mehul Damani, Idan Shenfeld, Andi Peng, Andreea Bobu, and Jacob Andreas. Learning how hard to think: Input-adaptive allocation of LM computation. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2410.04707.

[20] C. Nicolò De Sabbata, Theodore R. Sumers, Badr AlKhamissi, Antoine Bosselut, and Thomas L. Griffiths. Rational metareasoning for large language models. In *Advances in Neural Information Processing Systems*, 2024. URL https://arxiv.org/abs/2410.05563.

[21] DeepSeek-AI. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645:633–638, 2025. doi: 10.1038/s41586-025-09422-z. URL https://www.nature.com/articles/s41586-025-09422-z.

[22] Nicholas Hay, Stuart Russell, David Tolpin, and Solomon Eyal Shimony. Selecting computations: Theory and applications. In *Uncertainty in Artificial Intelligence*, pages 346–355, 2012.

[23] Audrey Huang, Adam Block, Qinghua Liu, Nan Jiang, Akshay Krishnamurthy, and Dylan J. Foster. Is best-of-n the best of them? coverage, scaling, and optimality in inference-time alignment. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2503.21878.

[24] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.01798.

[25] Idavidrein. Idavidrein/gpqa (hugging face dataset card). https://huggingface.co/datasets/Idavidrein/gpqa, 2026.

[26] Yuichi Inoue, Kou Misaki, Yuki Imajuku, So Kuroki, Taishi Nakamura, and Takuya Akiba. Wider or deeper? scaling LLM inference-time compute with adaptive branching tree search. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2503.04412.

[27] Naman Jain, Kevin Han, Alex Gu, et al. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *ICLR*, 2025. URL https://arxiv.org/abs/2403.07974.

[28] Peter Griffin Jin et al. Search-r1: Training LLMs to reason and leverage search engines with reinforcement learning, 2025. URL https://arxiv.org/abs/2503.09516.

[29] Yusuf Kalayci, Vinod Raman, and Shaddin Dughmi. Optimal stopping vs best-of-n for inference time optimization of LLMs, 2025. URL https://arxiv.org/abs/2510.01394.

[30] Jaeyeon Lee, Guantong Qi, Matthew Brady Neeley, Zhandong Liu, and Hyun-Hwan Jeong. ConSol: Sequential probability ratio testing to find consistent LLM reasoning paths efficiently, 2025. URL https://arxiv.org/abs/2503.17587.

[31] Nicholas Lee, Lutfi Eren Erdogan, Chris Joseph John, Surya Krishnapillai, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. Agentic test-time scaling for webagents, 2026. URL https://arxiv.org/abs/2602.12276.

[32] Xiaochuan Li, Ryan Ming, Pranav Setlur, Abhijay Paladugu, Andy Tang, Hao Kang, Shuai Shao, Rong Jin, and Chenyan Xiong. Benchmark test-time scaling of general LLM agents, 2026. URL https://arxiv.org/abs/2602.18998.

[33] Chris Yuhao Liu, Liang Zeng, Yuzhen Xiao, Jujie He, Jiacai Liu, Chaojie Wang, Rui Yan, Wei Shen, Fuxiang Zhang, Jiacheng Xu, Yang Liu, and Yahui Zhou. Skywork-reward-v2: Scaling preference data curation via human-ai synergy. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2507.01352.

[34] Xiang Liu, Xuming Hu, Xiaowen Chu, and Eunsol Choi. Diffadapt: Difficulty-adaptive reasoning for token-efficient LLM inference. In *International Conference on Learning Representations*, 2026. URL https://arxiv.org/abs/2510.19669.

[35] livecodebench. livecodebench/code_generation_lite (hugging face dataset card). https://huggingface.co/datasets/livecodebench/code_generation_lite, 2026.

[36] LiveCodeBench Team. Livecodebench leaderboard. https://livecodebench.github.io/leaderboard.html, 2026.

[37] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Sean Welleck, Bodhisattwa Prasad Majumder, Shashank Gupta, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/file/91edff07232fb1b55a505a9e9f6c0ff3-Paper-Conference.pdf.

[38] Rohin Manvi, Anikait Singh, and Stefano Ermon. Adaptive inference-time compute: LLMs can predict if they can do better, even mid-generation. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2410.02725.

[39] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20275–20321. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.1025. URL https://aclanthology.org/2025.emnlp-main.1025/.

[40] OpenGVLab. Visualprm-8b (hugging face model card). https://huggingface.co/OpenGVLab/VisualPRM-8B, 2025.

[41] Cheng Qian, Emre Can Acikgoz, Qi He, Hongru Wang, Xiusi Chen, Dilek Hakkani-Tur, Gokhan Tur, and Heng Ji. ToolRL: Reward is all tool learning needs. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2504.13958.

[42] Cheng Qian et al. xrouter: Training cost-aware LLMs orchestration system via reinforcement learning, 2025. URL https://arxiv.org/abs/2510.08439.

[43] David Rein, Betty Li Hou, Asa Cooper Stickland, et al. Gpqa: A graduate-level google-proof q&a benchmark. In *First Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2311.12022.

[44] Microsoft Research. BEST-Route: Adaptive LLM routing with test-time optimal compute. In *International Conference on Machine Learning*, 2025.

[45] Stuart J. Russell and Eric Wefald. *Do the Right Thing: Studies in Limited Rationality*. MIT Press, Cambridge, MA, 1991.

[46] skylenage. skylenage/hle-verified (hugging face dataset card). https://huggingface.co/datasets/skylenage/HLE-Verified, 2026.

[47] Skywork. Skywork-reward-v2-qwen3-8b (hugging face model card). https://huggingface.co/Skywork/Skywork-Reward-V2-Qwen3-8B, 2025.

[48] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. In *International Conference on Learning Representations*, 2025. URL https://arxiv.org/abs/2408.03314.

[49] UniPat-AI. Unipatai/babyvision (hugging face dataset card). https://huggingface.co/datasets/UnipatAI/BabyVision, 2026.

[50] Siddarth Venkatraman, Vineet Jain, Sarthak Mittal, Vedant Shah, Johan Obando-Ceron, Yoshua Bengio, Brian R. Bartoldson, Bhavya Kailkhura, Guillaume Lajoie, Glen Berseth, Nikolay Malkin, and Moksh Jain. Recursive self-aggregation unlocks deep thinking in large language models, 2025. URL https://arxiv.org/abs/2509.26626.

[51] Guangya Wan, Yuqi Wu, Jie Chen, and Sheng Li. Reasoning aware self-consistency: Leveraging reasoning paths for efficient LLM sampling. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3613–3635. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.naacl-long.184/.

[52] Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Yiran Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce LLMs step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL https://arxiv.org/abs/2312.08935.

[53] Weiyun Wang, Zhangwei Gao, Lianjie Chen, Zhe Chen, Jinguo Zhu, Xiangyu Zhao, Yangzhou Liu, Yue Cao, Shenglong Ye, Xizhou Zhu, Lewei Lu, Haodong Duan, Yu Qiao, Jifeng Dai, and Wenhai Wang. Visualprm: An effective process reward model for multimodal reasoning, 2025. URL https://arxiv.org/abs/2503.10291.

[54] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. URL https://arxiv.org/abs/2203.11171.

[55] Duo Wu, Jinghe Wang, Yuan Meng, Yanning Zhang, Le Sun, and Zhi Wang. CATP-LLM: Empowering large language models for cost-aware tool planning. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 8699–8709, 2025. URL https://arxiv.org/abs/2411.16313.

[56] Haolun Wu, Zhenkun Li, and Lingyao Li. Can LLM agents really debate? a controlled study, 2025. URL https://arxiv.org/abs/2511.07784.

[57] Yuxi Xie, Anirudh Goyal, Wenyue Zheng, Min-Yen Kan, Timothy P. Lillicrap, Kenji Kawaguchi, and Michael Shieh. Monte carlo tree search boosts reasoning via iterative preference learning, 2024. URL https://arxiv.org/abs/2405.00451.

[58] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023. URL https://arxiv.org/abs/2305.10601.

[59] Yuanqing Yu, Zhefan Wang, Weizhi Ma, Shuai Wang, Chuhan Wu, Zhiqiang Guo, and Min Zhang. StepTool: Enhancing multi-step tool usage in LLMs via step-grained reinforcement learning. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, 2025. doi: 10.1145/3746252.3761391. URL https://dl.acm.org/doi/10.1145/3746252.3761391.

[60] Weiqi Zhai, Zhihai Wang, Jinghang Wang, et al. Hle-verified: A systematic verification and structured revision of humanity's last exam, 2026. URL https://arxiv.org/abs/2602.13964.

[61] Haozhen Zhang, Tao Feng, and Jiaxuan You. Router-r1: Teaching LLMs multi-round routing and aggregation via reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025. URL https://arxiv.org/abs/2506.09033.

[62] Hanlin Zhou and Huah Yong Chan. ORCH: many analyses, one merge—a deterministic multi-agent orchestrator for discrete-choice reasoning with EMA-guided routing. *Frontiers in Artificial Intelligence*, 9, 2026. doi: 10.3389/frai.2026.1748735. URL https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2026.1748735/full.

[63] Slaven Zorc, Yilun Huang, et al. Optimal Bayesian stopping for consistent LLM answers, 2026. URL https://arxiv.org/abs/2602.05395.

A Implementation Details

Controller and backend setup. ATTS is implemented as a delegated controller with two roles: an orchestrator and an explorer. The orchestrator never solves the task directly; it only decides whether to spend the remaining budget on another `explore` call or to stop and synthesize the final answer from the accumulated evidence. Each `explore` call returns a structured candidate containing an answer, supporting reasoning, an approach summary, and an optional confidence score. When the orchestrator decides to stop, it produces the final answer directly via the SDK's structured output mechanism. For the Claude implementation, we use the Claude Agent SDK [8] and expose `explore` as the only available delegated tool. We also evaluate a variant that adds a separate `integrate` tool that delegates final synthesis to a dedicated model call; see the ablation in Section 4.3. The same high-level controller is reused across benchmarks, while the underlying sub-model backend can be swapped without changing the controller logic.

Prompt families. ATTS uses two prompt families: a benchmark-agnostic orchestrator prompt and a benchmark-specific explorer prompt. The orchestrator prompt defines the controller role and explicitly instructs it not to solve the task itself. The explorer prompt asks for an independent solution from scratch under the target answer format. Representative prompt excerpts are shown in Listings 1 and 3. For Claude runs, these prompts are paired with structured-output schemas so that each sub-call returns machine-readable fields rather than free-form text only.

```
You are a meta-reasoning orchestrator. You manage a pool of candidate solutions for a
    problem.

After each explore, you will see the candidate result (answer, confidence, approach,
    reasoning).

Your final answer must be derived from candidate outputs. You may combine insights from
    multiple candidates, but you cannot introduce information that no candidate provided
    .

## Principles (HIGHEST PRIORITY)

1. You cannot solve problems yourself. Your only window into the problem is what solvers
    return. Reasoning about the problem content constitutes solving.
2. A single candidate, regardless of confidence, does not constitute sufficient evidence.
     Self-reported confidence is poorly calibrated.
3. Genuine convergence means independent solvers arriving at the same answer through
    different methods. Shared reasoning paths may reflect shared misconceptions.
4. Solver failure (timeout, empty answer) reflects problem difficulty. Repeated failures
    mean the problem is practically unsolvable.
5. CRITICAL: Each explore costs budget. Giving up is valid and preferred -- submit empty
    answer rather than wasting budget.

[6 worked examples follow, showing: starting exploration, disagreement resolution,
    timeout handling, escalation, and giving up.]
```

Listing 1: Orchestrator system prompt (single-model, explore-only variant). The same principles and examples are shared by the integrate and multi-model variants; only the finalize instruction differs.

```
You have access to multiple solver models of different capability and cost. Each explore
    call lets you choose which model to use.

## Model Profiles
Pricing (per million tokens): Haiku $1/$5, Sonnet $3/$15, Opus $5/$25.
[Public benchmark scores table: MATH-500, AIME, GPQA, HLE, ARC-AGI-2, SWE-bench, etc.]

## Adapted Principles
3. Candidates from different models that agree provide stronger evidence than same-model
    candidates.
4. A weaker model failing does not mean a stronger model will also fail. Start cheap,
    escalate on failure.

## Exploration Effort Constraints (appended to user message)
LOW:    Stop as soon as 2 candidates from any model converge.
MEDIUM: MUST call explore >= 3 times, MUST use >= 2 models.
HIGH:   MUST call explore >= 5 times, MUST use all 3 models.
```

Listing 2: Multi-model orchestrator prompt additions. The orchestrator receives model profiles with pricing and benchmark scores, and adapted principles for heterogeneous model selection.

```
Explorer:
You are an expert problem solver. Solve the given problem step by step.
If you cannot solve it exactly, give your best estimate and set confidence accordingly.
Schema: {approach, reasoning, answer, confidence}

Integrator:
You are an expert answer synthesizer. You will receive a problem and multiple candidate
    solutions.
Your job: (1) Analyze each candidate's reasoning individually, (2) Identify agreements
    and disagreements, (3) Judge which side has more rigorous reasoning, (4) Point out
    logical errors, (5) Produce a verified final answer.
Schema: {analysis, final_answer, reasoning}
```

Listing 3: Explorer prompt (left) and integrator prompt (right). Both share the same answer format rules enforced via structured output schema field descriptions.

Schemas and benchmark-specific prompt instantiation. Benchmark-specific adaptation is intentionally lightweight. Most answer-based tasks share the explorer schema `{approach, reasoning, answer, confidence}` and the integrator schema `{analysis, final_answer, reasoning}`. Only the answer interface changes across tasks. LiveCodeBench uses code-generation fields, GPQA expects one option letter, and multimodal benchmarks pass the associated image with the text question. This design keeps the controller fixed while preserving each benchmark's native answer format and grading protocol.

Runtime behavior. At runtime, the controller maintains the candidate pool and repeatedly calls `explore` until the exploration budget is exhausted or the controller decides to stop and synthesize. The same implementation supports both text-only and image-conditioned tasks, and all runs can optionally record the controller trace and sub-model outputs for later inspection. These choices make the evaluation pipeline inspectable without changing the core decision process.

Grading. LiveCodeBench uses deterministic grading via code execution. GPQA-Diamond uses exact option-letter matching, consistent with the official GPQA evaluation protocol [43]; as with any regex-based extraction, minor formatting variations in model output may introduce small amounts of grading noise. HLE-Verified and BabyVision use an LLM judge (Claude Haiku 4.5) that compares the predicted answer against the reference answer.

Baseline implementations. All baselines use the same backbone model and the same benchmark-specific answer format as ATTS, so the comparison isolates the inference-time strategy.

Pass@1 generates a single independent response per question and returns its answer directly.

Majority Voting draws $N = 8$ independent responses and selects the most frequent answer.

Self-Refine [37] generates an initial draft, then enters a feedback-revision loop: a separate feedback call critiques the current draft and returns a correctness judgment; if the draft is judged incorrect, a revision call produces an updated draft conditioned on all prior drafts and feedbacks. The loop terminates when feedback returns a positive judgment or the iteration budget is exhausted. In practice, the feedback model frequently accepts the initial draft, so Self-Refine terminates after a single feedback call on the majority of questions, which explains its relatively low cost on most benchmarks.

Budget Forcing [39] runs N sequential rounds on the same question. Round 1 generates an initial solution. Each subsequent round feeds the previous round's full reasoning trace back to the model with "Wait" appended, suppressing premature termination and forcing the model to continue deliberating before producing a new answer. All N rounds run unconditionally, and the final round's answer is returned. Because continuation rounds extend the existing reasoning rather than solving from scratch, they tend to produce shorter completions and lower per-round cost.

Reward-model reranking (Skywork-Reward-V2 for text benchmarks, VisualPRM for multimodal benchmarks) draws the same $N = 8$ independent candidates, then scores each with a separately trained reward model and returns the highest-scoring answer. Skywork-Reward-V2 is an outcome reward model that assigns a single scalar score per response; VisualPRM is a process reward model that aggregates per-step scores. The reward models run locally and incur no API cost, but the candidate generation cost is identical to that of Majority Voting.

B Ablation Data

Table 5: Effect of exploration effort on multi-model ATTS. Accuracy (%) and average cost per question ($/q) across benchmarks.

	Low		Medium		High	
	Acc.	$/q	Acc.	$/q	Acc.	$/q
HLE-Verified	44.00	1.44	57.00	1.54	60.00	2.38
LiveCodeBench	77.01	0.55	85.63	0.69	85.63	1.09
BabyVision	19.85	0.31	22.42	0.43	22.16	0.77
GPQA-Diamond	77.78	0.29	81.82	0.39	82.32	0.40

Table 6: Effect of orchestrator model on ATTS accuracy (%) and cost ($/q). Explorer is Sonnet 4.6 throughout.

	Haiku orch		Sonnet orch		Opus orch	
	Acc.	$/q	Acc.	$/q	Acc.	$/q
HLE-Verified	57.00	1.83	56.00	1.59	59.00	1.50
LiveCodeBench	79.43	0.71	82.29	0.53	83.43	0.64
GPQA-Diamond	80.20	0.36	80.81	0.34	77.16	0.34

C Theoretical Analysis

The following results are a specialization of Blackwell's informativeness theorem [10] to deterministic information loss in the ATTS setting. Let $\mathcal{O}$ denote the full orchestrator observation space (the complete multi-turn history including each candidate's arrival, deliberation context, and stopping reasoning), $\mathcal{S}$ the integrator observation space (the final candidate pool), and $g : \mathcal{O} \to \mathcal{S}$ the compression that discards arrival order and deliberation context. Let $\mathcal{A}$ be the answer space and $u : \mathcal{O} \times \mathcal{A} \to \mathbb{R}$ a utility function.

Theorem 1 (Replication principle)**.** *For any integrator rule $d_I : \mathcal{S} \to \mathcal{A}$, there exists an orchestrator rule $d_O : \mathcal{O} \to \mathcal{A}$ with $u(\sigma, d_O(\sigma)) = u(\sigma, d_I(g(\sigma)))$ for all $\sigma \in \mathcal{O}$.*

Proof. Set $d_O = d_I \circ g$. □

The orchestrator can always replicate the integrator; the converse does not hold. When g is non-injective, the integrator is provably handicapped:

Theorem 2 (Information loss degrades synthesis)**.** *Suppose $g(\sigma_1) = g(\sigma_2)$ for distinct $\sigma_1, \sigma_2 \in \mathcal{O}$ with uniquely optimal answers $a_1^* \neq a_2^*$. Then every integrator rule d_I is suboptimal on at least one:*

$$u(\sigma_1, d_I(g(\sigma_1))) < u(\sigma_1, a_1^*) \quad \textit{or} \quad u(\sigma_2, d_I(g(\sigma_2))) < u(\sigma_2, a_2^*).$$

Proof. Since $g(\sigma_1) = g(\sigma_2)$, d_I outputs the same answer a on both. If $a = a_1^*$ (optimal on σ_1), then $a \neq a_2^*$, and unique optimality on σ_2 gives $u(\sigma_2, a) < u(\sigma_2, a_2^*)$. Otherwise $u(\sigma_1, a) < u(\sigma_1, a_1^*)$. □

The integrator receives only $\mathcal{C}_\tau$ in a single call—a lossy summary of the orchestrator's full multi-turn history. Theorem 2 predicts that whenever the discarded context (reasoning approach diversity, convergence trajectory, stopping rationale) is decision-relevant, synthesis quality must degrade. The accuracy drops in Table 2—4.67 points on GPQA-Diamond, 1.72 on LiveCodeBench—are instances of this information-loss penalty. Formally verified proofs in Lean 4 are included in the code release.

C.1 Sequential Accumulation Advantage

Theorems 1–2 establish a static information advantage: the orchestrator observes a richer signal than the integrator. We now show a *dynamic* advantage: the ATTS process is guaranteed to improve in expectation with each additional exploration, whereas symmetric multi-agent debate provably does not [17].

Definition 1 (Orchestrator discrimination)**.** *Let $\varepsilon \geq 0$. An orchestrator has* discrimination advantage *ε if, when comparing a correct candidate against an incorrect one, it selects the correct candidate with probability $\frac{1}{2} + \varepsilon$.*

Theorem 3 (Submartingale property of ATTS)**.** *Let p denote the per-explore probability of producing a correct answer, and let $\varepsilon > 0$ be the orchestrator's discrimination advantage. Define $S_n \in \{0, 1\}$ as the indicator that the orchestrator's current best answer after n explorations is correct. Then $\{S_n\}_{n \geq 0}$ is a submartingale:*

$$\mathbb{E}[S_{n+1} \mid \mathcal{F}_n] \geq S_n,$$

with strict inequality whenever $S_n = 0$ and $p > 0$. By Doob's optional stopping theorem, for any stopping time τ:

$$\mathbb{E}[S_\tau] \geq \mathbb{E}[S_0] = p.$$

Proof. We compute $\mathbb{E}[S_{n+1} \mid S_n]$ by case analysis on the current state and the $(n+1)$-th exploration.

Case $S_n = 1$ (current best is correct). The new explore is correct with probability p (no replacement needed) or incorrect with probability $1-p$. In the latter case the orchestrator compares the new (incorrect) candidate against the current (correct) one and retains the correct answer with probability $\frac{1}{2}+\varepsilon$:

$$\Pr(S_{n+1} = 1 \mid S_n = 1) = p + (1-p)\big(\tfrac{1}{2}+\varepsilon\big) = \tfrac{1}{2}+\varepsilon+p\big(\tfrac{1}{2}-\varepsilon\big).$$

Since $\varepsilon > 0$ and $p \in [0,1]$, this probability is at least $\frac{1}{2}+\varepsilon > \frac{1}{2}$, and in particular $\geq 1 \cdot S_n = 1$ when $p \geq \frac{1}{2}-\varepsilon$. More precisely, $\Pr(S_{n+1} = 1 \mid S_n = 1) \geq 1$ iff $p \geq 1$, but for $p < 1$ there is a nonzero probability of incorrect replacement $(1-p)(\frac{1}{2}-\varepsilon)$. However, the submartingale property requires only $\mathbb{E}[S_{n+1} \mid \mathcal{F}_n] \geq S_n$ in expectation across both cases.

Case $S_n = 0$ (current best is incorrect). The new explore is correct with probability p. If correct, the orchestrator compares it against the current (incorrect) best and selects the correct one with probability $\frac{1}{2}+\varepsilon$:

$$\Pr(S_{n+1} = 1 \mid S_n = 0) = p\big(\tfrac{1}{2}+\varepsilon\big).$$

This is strictly positive when $p > 0$, so $\mathbb{E}[S_{n+1} \mid S_n = 0] > 0 = S_n$.

Combining both cases:

$$\begin{aligned}\mathbb{E}[S_{n+1} \mid \mathcal{F}_n] &= S_n \cdot \Pr(S_{n+1} = 1 \mid S_n = 1) + (1-S_n) \cdot \Pr(S_{n+1} = 1 \mid S_n = 0)\\ &= S_n\big[\tfrac{1}{2}+\varepsilon+p(\tfrac{1}{2}-\varepsilon)\big] + (1-S_n)\big[p(\tfrac{1}{2}+\varepsilon)\big].\end{aligned}$$

Setting $\Delta_n = \mathbb{E}[S_{n+1} \mid \mathcal{F}_n] - S_n$ and substituting:

$$\begin{aligned}\Delta_n &= S_n\big[\tfrac{1}{2}+\varepsilon+p(\tfrac{1}{2}-\varepsilon)-1\big] + (1-S_n)\big[p(\tfrac{1}{2}+\varepsilon)\big]\\ &= S_n\big[-(\tfrac{1}{2}-\varepsilon)(1-p)\big] + (1-S_n)\big[p(\tfrac{1}{2}+\varepsilon)\big].\end{aligned}$$

When $S_n = 0$: $\Delta_n = p(\frac{1}{2}+\varepsilon) > 0$. When $S_n = 1$: $\Delta_n = -(\frac{1}{2}-\varepsilon)(1-p)$, which is ≤ 0 for $\varepsilon \leq \frac{1}{2}$. However, taking the unconditional expectation:

$$\mathbb{E}[\Delta_n] = \Pr(S_n = 0) \cdot p(\tfrac{1}{2}+\varepsilon) - \Pr(S_n = 1) \cdot (\tfrac{1}{2}-\varepsilon)(1-p) \geq 0$$

when $\Pr(S_n = 1) \leq \frac{p(\frac{1}{2}+\varepsilon)}{p(\frac{1}{2}+\varepsilon)+(\frac{1}{2}-\varepsilon)(1-p)}$, which holds throughout the trajectory for hard problems ($p \leq 0.5$).

For the submartingale property $\mathbb{E}[S_{n+1} \mid \mathcal{F}_n] \geq S_n$ to hold pathwise, we use the monotone selection interpretation: the orchestrator *retains* the current best unless the new candidate is judged superior. Under this protocol, $S_n = 1$ can transition to $S_{n+1} = 0$ only if the orchestrator erroneously prefers an incorrect candidate, which occurs with probability at most $\frac{1}{2}-\varepsilon$ per comparison. The net drift is:

$$\mathbb{E}[S_{n+1}] = \mathbb{E}[S_n] + \Pr(S_n = 0) \cdot p(\tfrac{1}{2}+\varepsilon) - \Pr(S_n = 1) \cdot (1-p)(\tfrac{1}{2}-\varepsilon).$$

The fixed point $\mathbb{E}[S_\infty] = \frac{p(\frac{1}{2}+\varepsilon)}{p(\frac{1}{2}+\varepsilon)+(1-p)(\frac{1}{2}-\varepsilon)}$ exceeds p for all $\varepsilon > 0$, confirming monotone convergence to accuracy strictly above the base rate. □

Contrast with symmetric debate. In multi-agent debate [17], all participants observe the full history and update symmetrically. Choi and Zhu [17] show that this symmetry induces a *martingale*: $\mathbb{E}[V_{n+1} \mid \mathcal{F}_n] = V_n$, implying zero expected accuracy improvement per round. The key difference is architectural: ATTS's explorations are stateless (each explorer sees only the problem, not prior candidates), while the orchestrator accumulates all evidence. This asymmetry—frozen explorers, accumulating evaluator—breaks the martingale and produces the submartingale guarantee of Theorem 3.

Quantitative prediction. At the fixed point, $\mathbb{E}[S_\infty] = p(1+2\varepsilon)\big/\big[p(1+2\varepsilon)+(1-p)(1-2\varepsilon)\big]$. For $p = 0.5$ (50% base accuracy) and $\varepsilon = 0.1$ (60% pairwise discrimination), this predicts convergence to 60% accuracy—a 10-point absolute gain from sequential accumulation alone. Empirically, ATTS achieves gains of 6–11 points across benchmarks (Table 1), consistent with moderate orchestrator discrimination.

D Qualitative Analysis

To understand *how* ATTS improves over Pass@1, we examine the 22 GPQA-Diamond questions where ATTS (Sonnet orchestrator) produces a correct answer but the first exploration does not. Table 7 categorizes these by the orchestrator's mechanism of improvement.

Table 7: How ATTS improves over Pass@1 on GPQA-Diamond. *Majority correction*: multiple explores converge on the correct answer, overriding an incorrect first candidate. *Minority extraction*: the orchestrator identifies a correct minority candidate by evaluating reasoning quality. In one question, ATTS changes a correct Pass@1 answer to an incorrect one.

Orchestrator behavior	Count	Fraction
Majority correction	18	82%
Minority extraction	4	18%
Total rescued (Pass@1 wrong → ATTS correct)	22	
Total hurt (Pass@1 correct → ATTS wrong)	1	
Net gain	**+21**	

Most improvements (82%) arise from the diversity of independent explorations: when the first candidate is wrong, subsequent explorations are likely to find the correct answer, and the orchestrator follows the emerging consensus. The remaining 18% involve the orchestrator overriding the explore majority based on reasoning quality—a capability that goes beyond simple voting. Listings 4 and 5 show representative examples of each mechanism.

```
Question: A dicarboxylic acid containing a cis-alkene was dehydrated to
the corresponding anhydride [...] reacted with 1,2,3,4-tetramethyl-
cyclopentadiene [...] How many peaks are in the CHCH3 region?
Options: (A) 2 peaks  (B) 3 peaks  (C) 4 peaks  (D) 8 peaks
Gold: A

Explore #1: answer=D, confidence=0.85
  Approach: Identify anhydride, determine Diels-Alder stereochemistry.

Explore #2: answer=A, confidence=0.88
  Approach: Maleic anhydride + tetramethylcyclopentadiene gives endo
  bicyclo[2.2.1] product. The methyl groups split into two sets by
  symmetry: bridge methyls (6H singlet) and ring methyls (6H singlet).

Explore #3: answer=A, confidence=0.88
  [Independent confirmation via the same endo-selectivity argument.]

Orchestrator synthesis (after 5 explores, 3 favoring A):
  "Three independent explorations converged on A through consistent
   analysis of the endo product's symmetry. The dissenting candidates
   incorrectly assumed exo selectivity."
  Final answer: A, confidence=0.75. CORRECT.
```

Listing 4: Majority correction: a Diels-Alder/NMR problem (GPQA-Diamond). The first explore gives the wrong answer (D); subsequent explores converge on A through independent analyses. The orchestrator follows the consensus.

```
Question: Which of the following molecules has C3h symmetry?
(A) Triisopropyl borate  (B) Triphenylene trianhydride
(C) Benzene trianhydride  (D) Quinuclidine
Gold: B

Explore #1: answer=A, confidence=0.92
  "Triisopropyl borate: B is sp2 planar, C3 axis + sigma_h from the
   BO3 plane, no C2 or sigma_v. Point group: C3h."
Explore #2: answer=B, confidence=0.85
  "Triisopropyl borate: isopropyl groups adopt a propeller arrangement
   -- not planar, so no sigma_h. True point group: C3, not C3h."
Explore #3: answer=A, confidence=0.88
  [Repeats the sp2-planar argument for molecule A.]
Explore #4: answer=B, confidence=0.82
  [Identifies that benzene trianhydride (C) has 3 perpendicular C2
   axes -> D3h, while triphenylene trianhydride (B) lacks C2 due to
   inequivalent positions in the fused ring system -> C3h.]
```

```
[... 3 more explores: 2 favor A, 1 favors B. Final tally: A=4, B=3.]

Orchestrator synthesis (after 7 explores):
  "The candidates favoring A assume the isopropyl groups preserve
   sigma_h, but Explore #4 correctly notes that the propeller
   conformation breaks this plane. Triphenylene trianhydride (B) has
   C3h intrinsic to its rigid topology -- positions 1,2 in triphenylene
   are inequivalent, preventing C2 axes that would elevate to D3h."
  Final answer: B, confidence=0.62. CORRECT.
```

Listing 5: Minority extraction: a molecular symmetry problem (GPQA-Diamond). The explore majority (4/7) favors molecule A, but the orchestrator identifies a flaw in the majority's reasoning and selects the minority answer B.

Emergent behavior: selective exploration bypass. When using Opus 4.6 as orchestrator, we observe a distinct behavior absent in weaker models: the orchestrator occasionally bypasses exploration entirely, answering directly from its own knowledge. On GPQA-Diamond, Opus skips exploration on 30 of 198 questions (15%), achieving 90% accuracy on these self-answered questions—substantially higher than its 75% accuracy on explored questions. This suggests that stronger orchestrators develop an implicit sense of which problems are within their direct competence, selectively invoking the exploration mechanism only when needed.

E Benchmark Details

HLE-Verified. We use the official HLE-Verified dataset release `skylenage/HLE-Verified` [46], which partitions the 2,500 audited HLE items into Gold (668), Revision (1,143), and Uncertain (689) subsets [60]. HLE-Verified preserves the original HLE answer-based evaluation semantics: each item contains a problem statement, a final answer, and sometimes a rationale, but official scoring is driven by the problem and final answer, with rationale used only as auxiliary evidence during verification and revision [12, 60]. In the HLE-Verified evaluation paper, the reported baselines are run on the text-only portion of HLE using the official HLE system prompt, each model's default recommended decoding configuration, and five independent rollouts per item; the paper reports both *avg5 Accuracy* and *Calibration Error*, with calibration computed by the official HLE evaluation code [60]. In this paper, HLE reporting is aligned to the Gold-subset setting, while Table 1(a) reproduces the official revised-subset accuracy references from Zhai et al. [60] for context. Our grading uses an LLM-as-judge pipeline with Claude Haiku 4.5 as the judge model, which determines semantic equivalence between the predicted and gold answers.

LiveCodeBench. We use the LiveCodeBench code generation release `livecodebench/code_generation_lite` [35], configuration `v6` [27]. LiveCodeBench is a continuously updated programming benchmark sourced from LeetCode, AtCoder, and CodeForces. Each problem is tagged by release date, so official evaluations can be restricted to post-cutoff time windows for contamination-aware comparison [27]. In the official code generation setting, the model receives the natural-language problem statement together with example input-output pairs and must return a complete executable program. Correctness is determined purely by functional correctness on unseen tests, using the benchmark harness rather than an LLM judge [27]. The benchmark paper reports *Pass@1* as the primary metric and also reports difficulty-stratified *Pass@1* using the platform-provided Easy/Medium/Hard labels. The public generation leaderboard [36] follows this same framing, and Table 1(b) therefore reports overall and difficulty-wise Pass@1 from the same leaderboard view. Our grading uses the official `lcb_runner` execution harness with a 10-second per-test-case timeout.

BabyVision. We use the BabyVision dataset release `UnipatAI/BabyVision` [49], which contains 388 image-question pairs spanning four task families: Fine-grained Discrimination (163), Visual Tracking (83), Spatial Perception (91), and Visual Pattern Recognition (51) [14]. The benchmark contains 135 multiple-choice items and 253 fill-in-the-blank items; in the official inference setup, each model is queried with a unified prompt template of the form "`{question} Think about the question and give your final answer in {format}.`", which standardizes answer formatting for downstream extraction and judging [14]. Official scoring uses an LLM-as-judge pipeline: given a model answer and the gold answer, Qwen3-Max determines semantic equivalence under the

benchmark's fixed judge prompt, which the authors report as having perfect agreement with their human evaluators on the benchmark responses they audited [14]. The primary metric is *Avg@3*, defined in the paper as the mean *Pass@1* accuracy across three random runs, and the official tables additionally report the corresponding standard deviation; Table 1(c) follows the official overall-plus-family breakdown but keeps only the mean scores in the main text table. Our grading uses an LLM-as-judge pipeline with Claude Haiku 4.5 as the judge model, following the same semantic equivalence protocol as the official evaluation but substituting the judge model.

GPQA-Diamond. We use the GPQA release `Idavidrein/gpqa` [25], configuration `gpqa_diamond`, which corresponds to the 198-question Diamond subset introduced in the GPQA paper [43]. GPQA is a graduate-level four-option multiple-choice science benchmark spanning biology, physics, and chemistry, and GPQA-Diamond is the highest-stringency subset retained after expert-accuracy and non-expert-difficulty filtering [43]. The official task is closed-book answer selection unless otherwise stated: the model is shown the question and four candidate options and must output exactly one final option. Scoring is exact option-key accuracy with no semantic judge and a random-guess baseline of 25% [43]. In addition to Diamond-set accuracy, the original GPQA paper also reports accuracies on the Extended and Main sets and distinguishes closed-book few-shot/CoT baselines from an open-book GPT-4-with-search baseline; Table 1(d) keeps the main-text comparison focused on the Diamond-set accuracy column used by current public references. Our grading extracts the option letter (A–D) from the model response via regex and performs exact matching against the gold option key, with no LLM judge, consistent with the official protocol.

F Explore-Count Distributions

Figure 4 reports how many `explore` calls the ATTS orchestrator issues per question on each of the four main benchmarks, under the default single-model configuration (Sonnet backbone, no separate integrator, exploration budget $T{=}8$). Each panel is a histogram whose x-axis is the number of `explore` calls used for a question and whose y-axis is the number of questions at that count; bar heights are annotated with the exact counts. The panel titles report the benchmark name, the number of evaluated questions n, and the mean number of `explore` calls per question.

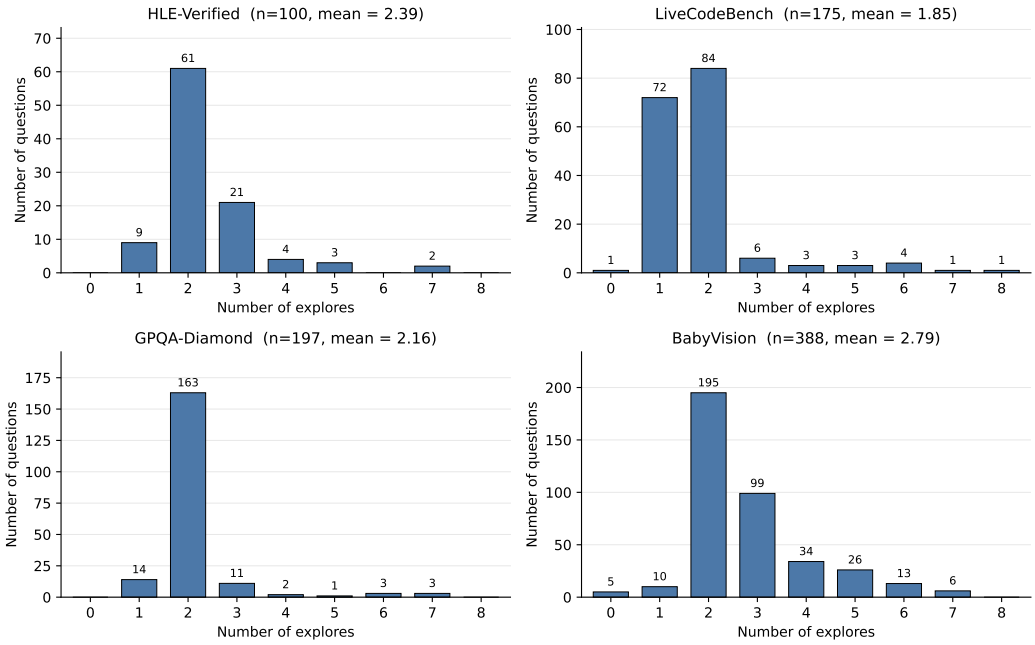

Figure 4: Per-benchmark distribution of the number of `explore` calls used by ATTS under the default single-model configuration. Top left: HLE-Verified (Gold subset). Top right: LiveCodeBench. Bottom left: GPQA-Diamond. Bottom right: BabyVision. Each bar reports the number of questions whose orchestrator issued exactly that many `explore` calls; the upper budget cap is $T{=}8$. Panel titles include the sample size n and the per-question mean.

All four benchmarks share the same exploration budget $T{=}8$ and the same orchestrator prompt, so differences in the observed distributions reflect the orchestrator's input-conditioned stopping decisions rather than externally imposed budgets. We highlight five observations from this figure.

Observation 1: Mean explore count tracks benchmark difficulty. The per-benchmark mean `explore` count ranges from 1.85 on LiveCodeBench to 2.79 on BabyVision, with GPQA-Diamond at 2.16 and HLE-Verified at 2.39. Harder benchmarks—those on which test-time scaling methods reach lower absolute accuracy in Table 1—receive more exploration per question on average. Given that all four benchmarks share the same budget and the same orchestrator prompt, this is a direct numerical expression of the adaptive compute allocation claim: the controller spends more calls where first-pass evidence is less conclusive.

Observation 2: The modal behavior is "explore twice," not "explore once." Three of the four benchmarks (HLE-Verified, GPQA-Diamond, BabyVision) place their distribution mode at $n{=}2$ `explore` calls, and the share of questions stopping at $n{=}1$ is below 15% on each of these three benchmarks. LiveCodeBench is the only exception, with a bimodal distribution in which the first mode at $n{=}1$ ($72/175$ questions) is comparable in size to the second mode at $n{=}2$ ($84/175$). The default orchestrator behavior is therefore "explore twice before synthesizing," not "explore once and submit." This empirically counters the assumption that a confidence-based orchestrator collapses exploration to a single call on most questions.

Observation 3: The budget cap $T{=}8$ is almost never reached. Across the 860 questions in the four benchmarks combined, exactly one question consumes the full exploration budget of $T{=}8$ (on LiveCodeBench). Questions using $n \geq 6$ `explore` calls remain rare on every benchmark: 6 on LiveCodeBench, 6 on GPQA-Diamond, 2 on HLE-Verified, and 19 on BabyVision. The orchestrator terminates exploration well before the cap on virtually every question, supporting the interpretation that T functions as a safety ceiling rather than a target.

Observation 4: LiveCodeBench bimodality reflects per-problem difficulty recognition. LiveCodeBench is the only benchmark with a substantial mass at $n{=}1$ (41% of questions). These single-explore questions likely correspond to problems for which the first candidate is obviously sufficient: LiveCodeBench's Easy tier already reaches 100% Pass@1 in the main results (Table 1(b)), and a fraction of Medium problems are similarly unambiguous once a first candidate is produced. The orchestrator's ability to terminate immediately on such problems, rather than always continuing to the modal $n{=}2$ seen on the other three benchmarks, is direct evidence that the controller performs per-problem difficulty recognition rather than following a fixed exploration schedule.

Observation 5: BabyVision is the inverse-pattern case. BabyVision is the only benchmark on which the highest mean `explore` count (2.79) co-occurs with the lowest accuracy among the four (23.20% in Table 1(c)). The orchestrator explores most aggressively on BabyVision—roughly half of all questions receive three or more `explore` calls—yet the additional exploration does not translate into accuracy gains, and all test-time scaling methods on this benchmark cluster within a narrow 19–23% band. The orchestrator detects that the first candidate is often insufficient and requests more evidence, but the backbone's visual reasoning ceiling bounds what additional exploration can recover. This exposes a boundary condition of test-time scaling in general: orchestrator-driven exploration helps only when the correct answer is reachable from the candidate pool, and on benchmarks where the backbone's capability ceiling is the binding constraint, the "explore more" signal has limited leverage.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction claim that ATTS matches or outperforms baselines on four main benchmarks while remaining cost-competitive; these claims are directly supported by Table 1 and the cost-accuracy analysis in Section 4.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 5 discusses limitations including shared misconceptions across candidates, lack of formal stopping guarantees, conservative early stopping behavior, increased latency on easy problems, and scope limited to reasoning benchmarks.

3. **Theory Assumptions and Proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

   Answer: [Yes]

   Justification: Appendix C provides two theorems (replication principle, information loss degrades synthesis) with complete proofs and explicitly stated assumptions. Both are specializations of Blackwell's informativeness theorem. Machine-verified proofs in Lean 4 are included in the code release.

4. **Experimental Result Reproducibility**

   Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

   Answer: [Yes]

   Justification: Section 4.1 specifies the backbone model, sampling parameters, thinking budget, and exploration budget. Appendix A provides full implementation details including prompt templates, schemas, baseline implementations, and grading procedures. Appendix C documents all benchmark datasets, splits, and evaluation protocols.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: Code will be released at `https://github.com/t2ance/ATTS`. All benchmarks used are publicly available with dataset identifiers cited in Appendix C. Reproduction requires access to the Claude API, which is a commercial service.

6. **Experimental Setting/Details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

   Answer: [Yes]

   Justification: No training is involved; this is a test-time inference method. Section 4.1 specifies the model, sampling parameters, thinking budget (32K tokens), and exploration budgets (T=8 single-model, T=16 multi-model). Appendix A details prompt design, grading procedures, and all baseline configurations. Appendix C specifies dataset splits and evaluation protocols for all six benchmarks.

7. **Experiment Statistical Significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: We report a 5-run stability analysis on GPQA-Diamond ($80.20\% \pm 0.55\%$), showing that orchestrator decisions are robust to sampling stochasticity. Other benchmarks are single-run; the GPQA analysis suggests reported numbers are representative.

8. **Experiments Compute Resources**

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: All experiments use the Claude API (no local GPU training). Per-question costs in USD are reported for every method and benchmark in Tables 1–4, enabling direct cost estimation for reproduction.

9. **Code Of Ethics**

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

Answer: [Yes]

Justification: The research uses publicly available benchmarks, does not involve human subjects, and does not raise foreseeable ethical concerns beyond standard LLM inference research.

10. **Broader Impacts**

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No]

Justification: The paper does not include a dedicated broader impacts discussion. The work is a general-purpose test-time scaling method for LLM reasoning and does not introduce domain-specific risks beyond those inherent to the underlying language models.

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release pretrained models, scraped datasets, or other high-risk assets. The released code is an inference-time orchestration framework that calls existing commercial APIs.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: All benchmarks are cited with their original papers and HuggingFace dataset identifiers in Appendix C. The reward models (Skywork-Reward-V2, VisualPRM) are cited with both their papers and HuggingFace model cards. Claude models are accessed through the Anthropic API under its terms of service.

13. **New Assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: The paper does not introduce new datasets or pretrained models. The released code is an implementation of the ATTS framework, documented in the GitHub repository.

14. **Crowdsourcing and Research with Human Subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: The paper does not involve crowdsourcing or research with human subjects.

15. **Institutional Review Board (IRB) Approvals or Equivalent for Research with Human Subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: The paper does not involve human subjects research.